

Touying + Navigator

Structural Transitions · Per-slide Titles via slide-level: 3

Background & Motivation

Why Structure Matters

Existing Approaches

Background & Motivation

Why Structure Matters

Existing Approaches

The audience perspective

Structured presentations help audiences track where they are in a talk. Navigator generates transition slides automatically from the document outline.

- Section transitions: show the full plan
- Subsection transitions: highlight current position
- Slides stay in sync without manual effort

A key observation

“A well-structured talk is a talk the audience can follow even when they lose concentration for a moment.”

Background & Motivation

Why Structure Matters

Existing Approaches

Polylux: show-rule-based transitions

Most Typst presentation packages handle navigation manually:

- **Polylux:** `show heading: h => render-transition(h, ...)`
- **Diatypst / typslides:** similar hook-based approach
- **Touying:** native section/subsection hooks → cleaner integration

Why native hooks are better

With Touying's hooks, `here()` already points to the heading at hook call time — no need to pass `h.location()` explicitly.

This makes `match-page-only`: `true` reliable for identifying the active entry in `progressive-outline`.

Design & Implementation

Implicit Slides via `slide-level: 3`

The Preamble Fallback

Design & Implementation

Implicit Slides via `slide-level: 3`

The Preamble Fallback

How the heading hierarchy works

Setting `slide-level: 3` in `config-common` changes the heading hierarchy:

Level	Syntax	Effect
1	<code>= Section</code>	Section transition (nav-transition-slide)
2	<code>== Subsection</code>	Subsection transition (nav-transition-slide)
3	<code>=== Title</code>	Content slide — no <code>#slide[]</code> needed

The document structure

<u>= Section</u>	// → transition slide
<u>== Subsection</u>	// → transition slide
<u>=== Slide title</u>	// → content slide (implicit)
Content here...	
<u>=== Next slide</u>	// → next content slide (implicit)
More content...	

Each === heading opens a new slide and its text appears in the preamble.

Design & Implementation

Implicit Slides via `slide-level: 3`

The Preamble Fallback

Preamble with a slide-specific title

This slide has a === heading, so its own title is shown above. The subsection title (“The Preamble Fallback”) is **not** shown.

The preamble logic:

```
let h3 = utils.current-heading(level: 3)
let title = if h3 != none { h3.body } else {
  let h2 = utils.current-heading(level: 2)
  if h2 != none { h2.body } else { none }
}
```

Preamble with a slide-specific title

This slide was created with an explicit `#slide[]` — no `===` heading.

The preamble above shows **“The Preamble Fallback”** (the current subsection title), because no level-3 heading is active on this page.

Explicit `#slide[]` blocks can always be mixed with implicit `===` slides.

Results & Conclusion

Summary

Conclusion

Results & Conclusion

Summary

Conclusion

What this example demonstrates

- `slide-level: 3` enables heading-driven slide creation without `#slide[]`
- `===` headings appear as per-slide titles in the preamble
- Slides without a `===` heading (explicit `#slide[]`) fall back to `==`
- `level-3-mode: "none"` keeps `===` headings out of transition outlines
 - they are an intra-slide concern, invisible to progressive-outline

Results & Conclusion

Summary

Conclusion

Zero boilerplate

**slide-level: 3 + progressive-outline + per-slide === titles
→ fully structured presentations with zero boilerplate**

See also: `polylux_progressive_outline.typ` for the Polylux approach.